Reporte Técnico de Actividades Práctico-Experimentales Nro. 004

1. Datos de Identificación del Estudiante y la Práctica

Nombre del estudiante(s)	Tayron Morales Mateo Rojas Leonardo Sánchez
Asignatura	Sistemas distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	4
Título de la Práctica	Implementar un servicio REST con Node.js
Nombre del Docente	José O. Guamán Q.
Fecha	13 de octubre de 2026 16 de octubre de 2026
Horario	Martes 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	1 horas

2. Objetivo(s) de la Práctica

Implementar un sistema de asignación de pedidos que integre comunicación entre procesos (colas IPC) y comunicación en red (sockets TCP), y utilizar un agente de IA local (Ollama) provisto por el docente para la asignación de tareas, evaluando su comportamiento frente a la carga y la distancia.

Objetivos específicos

- Implementar comunicación entre procesos usando colas (IPC).
- Desarrollar comunicación entre computadoras mediante sockets TCP.
- Integrar un agente de decisión basado en IA usando Ollama.
- Simular un sistema real de asignación de pedidos (tipo delivery).
- Evaluar el comportamiento del sistema en función de la carga y la distancia.
- Analizar ventajas de usar IA frente a métodos tradicionales (reglas fijas).

3. Materiales y reactivos

- Guia practic
- 2 o más computadoras (misma red WiFi/LAN)
- Router o red local

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación (Python, java sprint boot, node)
- IDE de programación
- Ollama

5. Procedimiento / Metodología

Primero antes de empezar se nos proporcionó un repositorio en GitHub del docente encargado, el cual cuenta con el código de la práctica y se lo fue analizando y revisando para entender su funcionamiento.

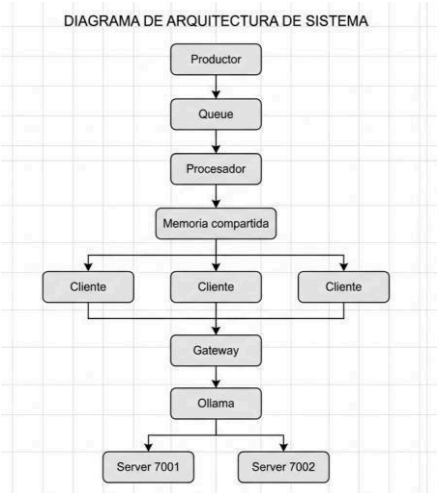


Fig 1: Arquitectura actual

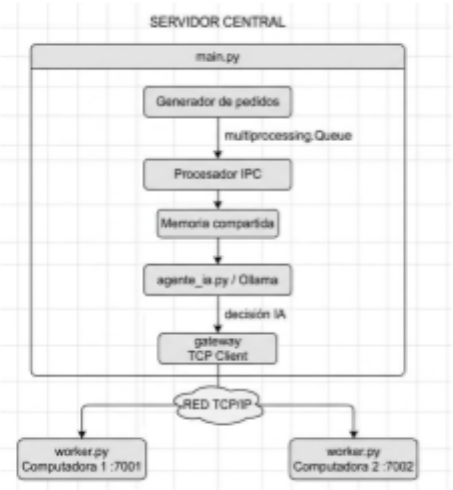
Esta es la arquitectura del código en GitHub y en la siguiente tabla se muestran las funcionalidades de cada archivo original.

Script actual	Ubicación	Función
app.py	cliente_system/	Es el punto de inicio actual del sistema. Inicia el sistema IPC y crea tres hilos clientes.
cliente.py	cliente_system/	Lee datos de la memoria compartida y los envía mediante TCP al

		Gateway.
main.py	cliente_system/ipc_system/	Crea la cola IPC, la memoria compartida y los procesos productor/procesador.
productor.py	ipc_system/controls/	Genera datos simulados de temperatura y vibración y los coloca en la cola.
procesador.py	ipc_system/controls/	Extrae datos de la cola, calcula una condición de alerta y guarda el resultado en memoria compartida.
gateway.py	gateway-system/	Recibe datos de los clientes, consulta a Ollama y decide a qué servidor enviar los datos.
server.py	servidores-system/	Actúa como nodo servidor. Recibe los datos mediante TCP y los muestra en pantalla.

Lo que nosotros debemos realizar es una nueva arquitectura con su respectivo código usando un agente de IA local Ollama. Con ayuda de la IA se fue modificando y entendiendo el nuevo código con sus nuevas funcionalidades como el productor.py generara pedidos, el procesador.py procesara los pedidos, el gateway.py sera el gateway de distribución, se creara el agente_ia.py para conectar con la parte del API de la universidad, el cliente.py podra eliminarse o integrarse en el gateway, el app.py se debe poder convertirse en main.py y el ipc_system/main.py se podrá integrar en el main.py.

Con los cambios hechos la arquitectura cambia y tendría la siguiente estructura.



integrar en el main.py.

Con los cambios hechos la arquitectura cambia y tendría la siguiente estructura.

El sistema implementa una arquitectura distribuida que integra comunicación entre procesos (IPC), memoria compartida, sockets TCP y un agente de inteligencia artificial para clasificar y distribuir solicitudes hacia servidores trabajadores.

Inicialmente, el archivo main.py coordina la ejecución de dos servidores en los puertos 7001 y 7002, un gateway y una aplicación cliente. Esta última utiliza multiprocessing para crear una cola IPC y memoria compartida que permiten la comunicación entre procesos independientes.

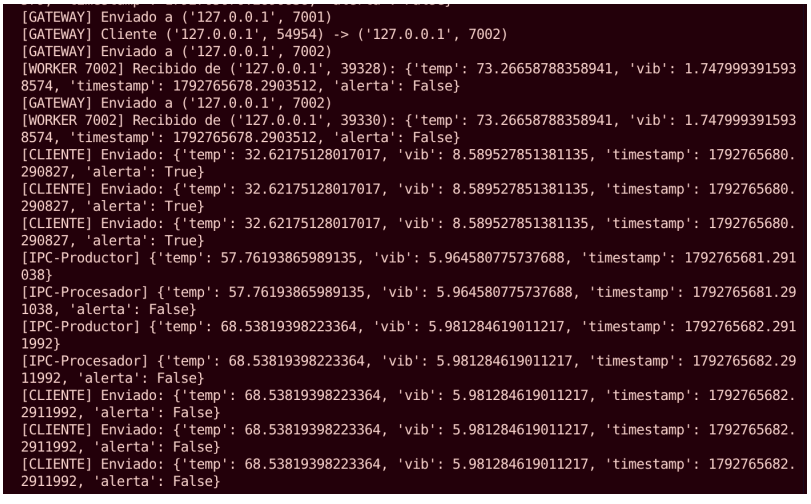
El módulo productor.py genera datos simulados de temperatura, vibración y una marca temporal, que son enviados a una cola IPC. Posteriormente, procesador.py analiza estos valores y activa una alarma cuando la temperatura supera los 75 grados o la vibración es mayor que 7. Los resultados se almacenan en memoria compartida para su posterior consulta.

A continuación, cliente.py recupera los datos procesados y los envía en formato JSON mediante sockets TCP al gateway, que escucha en el puerto 6000. Este utiliza agente_ia.py para comunicarse mediante una API HTTP con el servicio de inteligencia artificial disponible en http://10.10.180.10:3000 correspondiente a Open WebUI, mediante el cual se accede al modelo de Ollama para clasificar los mensajes como CRÍTICO o NORMAL

Según la respuesta del agente de IA, el gateway selecciona el servidor al que enviará la información. Si el servicio de inteligencia artificial falla o no devuelve una respuesta válida, se utiliza el algoritmo Round Robin como mecanismo alternativo de distribución.

Finalmente, los servidores implementados en server.py reciben los mensajes JSON en los puertos 7001 y 7002 y muestran su contenido en consola, permitiendo verificar la comunicación y distribución de los datos.

De esta manera, el sistema demuestra la integración de mecanismos IPC, comunicación TCP y clasificación mediante inteligencia artificial en un entorno distribuido.



6. Resultados

- Correcta comunicación entre procesos (IPC).
- Envío de datos entre computadoras.
- Decisiones automáticas del agente IA.
- Distribución eficiente de pedidos
- La implementación esta en este enlace: https://github.com/joseguaman1/distribuidos.git

7. Preguntas de Control

- **¿Qué diferencia hay entre la comunicación IPC (cola de procesos) y la comunicación por sockets TCP?**

  IPC nos permite la comunicación entre los procesos en una computadora, usando los mecanismos como multiprocessing queque, sockets TCP que nos permite comunicar procesos que son ubicados en computadoras diferentes mediante la red, IPC que trabaja localmente.

- **¿Por qué el agente IA y el generador de pedidos pueden ejecutarse en computadoras distintas sin compartir memoria?**

  Se comunican mediante los mensajes que son enviados a la red, con el generador envía los datos serializados, como por ejemplo en formato JSON

- **¿Qué información debe viajar en el mensaje para que el agente decida correctamente? ¿Qué pasaría si faltara la carga del nodo?**

  Se debe incluir la información necesaria para tomar la decisión:

  ```
  {
   "temp": 85,
   "vib": 8,
   "timestamp": 1720000000,
   "alerta": true
  }
  ```

  Puede incluir a la carga de cada nodo, su distancia, la prioridad y cada identificador de cada pedido, si se falta la carga del nodo, la IA no se podrá evaluar.

- **¿Qué ocurre si un worker se desconecta mientras hay pedidos en la cola?**

  EL envío del TCP fallara, el gateway se debe detectar el error, evitar que el programa se detenga para reenviar el pedido a otro worker que es disponible para conservarlo en una cola para reintentar, el gateway registra el error con el round robin se utiliza cuando falla la decisión de IA.

- **¿En qué casos un agente de IA aporta valor frente a reglas fijas y en cuáles no?**

  La IA nos aporta un valor cuando existen varios factores y decisiones variables como la temperatura, vibración, distancia, carga y la prioridad, cada reglas fijas son preferibles cuando las condiciones son simples, conocidos y deterministas son rápidas, fáciles de probar y no dependen de una conexión de modelo.

8. Conclusiones

Redacte las principales conclusiones de la práctica, vinculándolas con el objetivo planteado.

9. Declaración de IA

Redacte las principales conclusiones de la práctica, vinculándolas con el objetivo planteado.

10. Bibliografía / Referencias

- Semana 4. Guaman, Jose. 2026